

MINI-PROJECT REPORT

ON

**“HireForge AI: An AI-Powered Platform for Intelligent
Resume Screening and Semantic Job Matching”**

FOR PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE
MINI-PROJECT
OF T.E. E&CE – 2019 COURSE
(SAVITRIBAI PHULE PUNE UNIVERSITY)

By

Harshal Kale	35126
Prathmesh Kurhade	35135
Atharv Lalage	35136

PROJECT GUIDE

Ms. S.C Nahatkar



**DEPARTMENT OF
ELECTRONICS AND COMPUTER ENGINEERING
SCTR'S PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE – 43**

ACADEMIC YEAR: 2025 - 2026

Department of Electronics and Computer Engineering
SCTR's Pune Institute of Computer Technology, Pune – 43

CERTIFICATE

This is to certify that the Mini-Project Report entitled “HireForge AI: An AI-Powered Platform for Intelligent Resume Screening and Semantic Job Matching” has been successfully completed by

Harshal Kale
Prathmesh Kurhade
Atharv Lalage

Exam No. _____
Exam No. _____
Exam No. _____

Is a bonafide work carried out by them under the supervision of Ms. S.C Nahatkar and it is approved for the partial fulfillment of the requirements for the Mini-Project of T.E. E&CE, 2019 Course of the Savitribai Phule Pune University, Pune.

Ms. S.C Nahatkar
Project Guide
Dept. of E&CE

Dr. S. K. Moon
HOD, E&CE

Place: Pune
Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide Ms. S.C Nahatkar for her invaluable guidance, unwavering support, and expert mentorship throughout the development of this project. Her deep knowledge of AI systems and software engineering was instrumental in shaping the architectural decisions and technical direction of HireForge AI.

We are deeply thankful to Dr. S. K. Moon for his consistent encouragement, constructive feedback, and insightful suggestions that helped us refine both the implementation and the presentation of this work.

We extend our gratitude to the faculty and staff of the Department of Electronics and Computer Engineering, Pune Institute of Computer Technology, for providing the computational infrastructure, laboratory access, and academic environment necessary for this project.

We acknowledge the open-source community and the developers behind the tools and frameworks that made this project possible: React, FastAPI, Sentence-Transformers, scikit-learn, pdfplumber, Framer Motion, TailwindCSS, and the Groq API platform. We also thank NeonDB for providing free-tier serverless PostgreSQL hosting.

Finally, we express our heartfelt thanks to our families and friends for their moral support and patience throughout the duration of this project.

Harshal Kale – 35126

Prathmesh Kurhade – 35135

Atharv Lalage – 35136

ABSTRACT

The increasing volume of job applications in modern recruitment has created a significant challenge for recruiters to efficiently screen and shortlist suitable candidates. Traditional Applicant Tracking Systems (ATS) rely on keyword matching, which fails to distinguish between candidates of varying proficiency levels — a resume mentioning "competitive programming" is treated identically regardless of whether the candidate solved 300 problems or 1,000 problems at the Codeforces Specialist level. This inability to reason about qualitative differences leads to frequent mis-ranking of candidates.

This project presents HireForge AI — an AI-powered recruitment platform that implements a Two-Stage Semantic Matching Pipeline to address these limitations. In Stage 1, resume and job description documents are parsed using pdfplumber, processed through a variant-aware NLP skill extraction engine, and converted into 384-dimensional dense vector embeddings using the all-MiniLM-L6-v2 Sentence-BERT model. Cosine similarity between these embeddings provides a fast, local semantic score. A keyword-based skill match ratio is computed in parallel, and the two signals are combined using a weighted formula ($0.6 \times \text{semantic} + 0.4 \times \text{skill ratio}$) with a square-root curve for intuitive score distribution.

In Stage 2, the top 10 candidates from Stage 1 are sent in a single batched API call to the Groq Llama 3.3 70B Large Language Model. The LLM acts as a virtual senior recruiter, reasoning about proficiency hierarchies (Codeforces ratings, CodeChef star levels, project depth, internship quality) and returning a 0–100 score with a one-line human-readable justification for each candidate. The final ranking is computed as a weighted fusion: $0.9 \times \text{LLM score} + 0.1 \times \text{embedding score}$.

The platform further integrates an Ethical AI Layer featuring identity masking (names replaced with neural placeholders like CANDIDATE_VECTOR_8241), PII redaction via regex and NLP passes, and a Synaptic Audit Log for compliance tracking. The frontend is built with React 18, Vite, TailwindCSS, and Framer Motion, featuring a Holographic Neural Dossier for candidate visualization. The backend uses FastAPI with SQLAlchemy ORM and JWT authentication, hosted on NeonDB serverless PostgreSQL.

Experimental results with real resumes demonstrate that the hybrid ranking correctly identifies stronger candidates that embedding-only systems mis-rank, while maintaining graceful degradation (falling back to embedding-only scores if the LLM API is unavailable). The system processes a typical job with 15 candidates in under 3 seconds end-to-end.

LIST OF FIGURES

Figure No.	Figure Caption	Page No.
2.1	System Architecture — Block Diagram	17
2.2	Two-Stage Ranking Pipeline — Flowchart	18
3.1	Database Schema — Entity Relationship Diagram	20
3.2	384-Dimensional Vector Space (t-SNE Projection)	22
3.3	Ethical AI Layer — Identity Masking Flow	22
4.1	Recruiter Dashboard — Job Creation Interface	27
4.2	Resume Upload and Parsing Interface	27
5.1	Embedding-Only vs. Hybrid (LLM) Ranking Comparison	29
5.2	Score Breakdown Heatmap Across Pipeline Stages	29

LIST OF TABLES

Table No.	Title of Table	Page No.
1.1	Literature Survey Summary	13
2.1	Comparison: Keyword-Based ATS vs. HireForge AI	16
3.1	Software and Technology Stack	19
4.1	Database Tables and Purpose	21
4.2	Key API Endpoints	22
4.3	Embedding Model Specifications	23
5.1	Skill Canonicalization Mappings (Sample)	25
5.2	LLM Prompt Design Parameters	26
5.3	Test Cases and Results	27
6.1	Ranking Results — Embedding vs. Hybrid	29
6.2	Score Breakdown Per Pipeline Stage	30
6.3	Performance Benchmarks	30
6.4	Ethical Layer Functionality Test	31
7.1	Feature Comparison with Industry Tools	33

ABBREVIATIONS

Abbreviation	Full Form
ATS	Applicant Tracking System
NLP	Natural Language Processing
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
BERT	Bidirectional Encoder Representations from Transformers
PII	Personally Identifiable Information
ORM	Object-Relational Mapping
SPA	Single Page Application
PDF	Portable Document Format
SQL	Structured Query Language
PK	Primary Key
FK	Foreign Key
t-SNE	t-Distributed Stochastic Neighbor Embedding
UI	User Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
HMR	Hot Module Replacement

CHAPTER 1

Introduction

The global recruitment industry processes over 250 million job applications annually, with large corporations receiving an average of 250 resumes per open position. This volume has made manual screening impractical, leading to widespread adoption of Applicant Tracking Systems (ATS). However, the fundamental technology powering most ATS platforms remains keyword-based string matching — a method that is fast but semantically blind.

Keyword matching treats all mentions of a skill identically. A resume stating "I have basic knowledge of machine learning" and one stating "I architected and deployed production ML pipelines serving 10 million users" would receive the same relevance score for the keyword "machine learning." This inability to assess proficiency depth causes two critical failures: qualified candidates are rejected because they phrase skills differently (synonym mismatch), and unqualified candidates are over-ranked because their resumes are optimized for keyword density.

The emergence of Transformer-based language models has introduced a new paradigm: semantic understanding. Models like BERT and its distilled variants can encode text into dense vector representations that capture contextual meaning, not just surface-level word overlap. Cosine similarity between these vectors provides a measure of semantic closeness that transcends exact keyword matching. However, even embedding-based similarity has a limitation: it captures semantic closeness but not evaluative judgment. An embedding model can tell that two resumes are "about competitive programming," but it cannot reason that one candidate is a Codeforces Specialist (rating 1563) with 1,000 problems solved while another is a 2-Star CodeChef with 300 problems. This distinction requires the kind of contextual reasoning that only a Large Language Model (LLM) can provide.

HireForge AI addresses this gap by implementing a Two-Stage Semantic Matching Pipeline that combines the speed of vector embeddings with the reasoning capability of an LLM, producing candidate rankings the way a human senior recruiter would — understanding proficiency levels, not just keyword hits.

1.1 Relevance

HireForge AI integrates core concepts from the Electronics and Computer Engineering curriculum across multiple domains. In the area of Artificial Intelligence and Machine Learning, the system employs LLM inference, prompt engineering, and structured output generation using Groq Llama 3.3 70B for candidate re-ranking and interactive evaluation.

In Data Structures and Algorithms, 384-dimensional vector indexing utilizes dense embedding spaces while cosine similarity implements linear algebraic retrieval for fast semantic comparison.

The project applies Database Systems concepts through PostgreSQL relational schema design with JSON columns for high-dimensional embedding storage. Software Engineering principles are demonstrated through the modular two-stage FastAPI pipeline architecture with clear separation of concerns between parsing, embedding, and LLM layers. Information Security is addressed through JWT-based stateless authentication, bcrypt password hashing, and the Ethical AI Layer performing PII redaction ensuring recruiters perform initial evaluations without access to candidate identity.

1.2 Motivation

The motivation for this project emerged from a concrete observation during competitive programming recruitment drives. When testing early embedding-only prototypes with real resumes, a consistent failure pattern was observed:

Candidate A, with 300 solved problems and a 2-Star CodeChef rating, ranked higher than Candidate B, who had 1,000+ solved problems, a Codeforces Specialist rating (1563), and a 3-Star CodeChef rating (1636). The embedding model ranked Candidate A first because his resume was heavily framed around competitive programming keywords, while Candidate B buried his stronger credentials at the bottom of his resume. The embedding captured keyword density, not credential quality.

This real failure case — where the stronger candidate was ranked lower because of resume formatting, not merit — motivated the addition of an LLM re-ranking stage. A language model, given the raw text of both resumes, could reason about the actual credentials (ratings, problem counts, contest achievements) and correctly rank the stronger candidate higher.

A secondary motivation was the ethical dimension. Unconscious bias in hiring based on names, gender, and demographic indicators is a well-documented problem. Building identity masking directly into the platform — as a middleware that intercepts data before it reaches the recruiter's screen — ensures that the initial evaluation is purely merit-based.

A thorough examination of existing literature and recruitment technology reveals four critical gaps: (1) keyword-based ATS systems cannot assess proficiency depth or qualitative differences in candidate credentials; (2) embedding-only systems are biased by resume formatting rather than actual merit; (3) no existing recruitment AI integrates an architectural identity masking layer for bias-free evaluation; (4) LLM-based re-ranking has

not been applied specifically to the recruitment domain with batched prompting and graceful degradation. These gaps collectively motivated the development of HireForge AI.

1.3 Problem Definition and Objectives

Recruitment screening is inherently complex, and existing AI-based tools either operate purely on keyword overlap (ATS) or fail to reason about credential quality differences (embedding-only systems). The core problem is: How can we build an AI-powered resume screening platform that accurately ranks candidates by assessing proficiency depth, not just keyword density, while maintaining ethical bias-free evaluation and operational resilience?

The objectives are:

1. To develop an AI-powered web platform for intelligent resume screening and semantic job matching using a two-stage pipeline.
2. To design a Stage 1 semantic filter using Sentence-BERT embeddings (all-MiniLM-L6-v2) combined with a variant-aware NLP skill extraction and canonicalization engine.
3. To implement a Stage 2 LLM re-ranker using Groq Llama 3.3 70B with batched prompting, proficiency hierarchy instructions, and graceful degradation fallback.
4. To integrate an Ethical AI Layer with identity masking, PII redaction via regex and NLP, automatic unmask on Fast-Track action, and an immutable Synaptic Audit Log.

1.4 Project Scope and Limitations

HireForge AI covers end-to-end processing of PDF and DOCX resumes, supporting job description creation, multi-candidate batch upload, two-stage semantic ranking, and ethical AI-gated candidate decision-making (Fast-Track / Reject). The platform provides automated skill extraction, semantic similarity scoring, LLM-based re-ranking with explanations, and an identity masking layer compliant with ethical hiring best practices.

Limitations include: Stage 1 embedding quality depends on the all-MiniLM-L6-v2 model and degrades for resumes with unconventional structure; Stage 2 LLM re-ranking introduces API latency (~0.8–1.5s) and requires an active internet connection; the system supports single-session design without cross-job candidate discovery; PII recognizers are optimized for standard English patterns and may miss highly obfuscated formats; and the 90/10 fusion weighting, while empirically validated, may require recalibration for specialized domains where embedding signals carry more domain-specific weight.

Table 1.1: Comparison — Keyword-Based ATS vs. HireForge AI

Capability	Traditional ATS	HireForge AI
Matching Method	Keyword string matching	Semantic embedding + LLM reasoning
Proficiency Assessment	None (all mentions equal)	LLM reasons about ratings, problem counts
Synonym Handling	Manual synonym lists	Automatic via 384-dim vector space
Explainability	Score only (black box)	LLM reason per candidate
Bias Mitigation	Not built-in	Identity masking + PII redaction
Fallback Handling	Single point of failure	Graceful degradation to embedding-only
Audit Trail	Limited logging	Immutable Synaptic Audit Log

Table 1.1: Comparison — Keyword-Based ATS vs. HireForge AI

CHAPTER 2

Literature Survey

2.1 Introduction

This chapter reviews existing literature across six key domains relevant to HireForge AI: semantic text embeddings, transformer-based language models, resume parsing and NLP, LLM-based re-ranking, bias mitigation in automated hiring, and two-stage retrieval architectures. The review identifies strengths, limitations, advantages, disadvantages, salient features, and applications of prior work, and establishes the research gaps that HireForge AI addresses.

Reimers and Gurevych introduced Sentence-BERT (SBERT) in 2019, demonstrating that siamese BERT networks fine-tuned on natural language inference tasks produce semantically meaningful sentence embeddings suitable for cosine similarity comparison [1]. The all-MiniLM-L6-v2 variant distills this capability into a 6-layer model with 384-dimensional output, achieving CPU inference under 50ms per document — the embedding backbone of HireForge AI's Stage 1.

Devlin et al. established transformer-based contextual embeddings as state-of-the-art for NLP tasks through BERT [2], while Roy et al. applied TF-IDF and keyword extraction to resume-JD matching, confirming that keyword-only systems lack semantic understanding and cannot assess proficiency depth [3]. Meta AI's LLaMA 3.3 70B model, accessed via the Groq API, provides strong instruction-following and domain-specific reasoning capabilities required for Stage 2 re-ranking [4].

Raghavan et al. demonstrated that blind screening using identity masking reduces demographic hiring bias by 25–46%, providing the empirical basis for HireForge AI's Ethical AI Layer [5]. Nogueira and Cho established two-stage retrieval + re-ranking as an effective paradigm for information retrieval [6], validating the architectural approach adopted by this project. The Vaswani et al. attention mechanism [15] underpins all transformer-based components used.

The literature reveals that while embedding-based semantic matching and LLM reasoning have been studied independently, no existing system combines them into a two-stage pipeline specifically for recruitment, with an integrated ethical AI layer featuring architectural identity masking. This project fills that gap.

2.2 Summary

Table 1.2: Literature Survey Summary

Sr.	Author / Source	Topic	Key Contribution	Limitation
1	Reimers & Gurevych (2019)	Sentence-BERT Embeddings	Introduced efficient sentence-level semantic similarity via siamese BERT fine-tuning	Does not perform evaluative reasoning; treats all semantically similar text equally
2	Devlin et al. (2019)	BERT: Deep Bidirectional Transformers	Established transformer contextual embeddings as NLP state-of-the-art	Large model size; not suitable for local inference without distillation
3	Roy et al. (2020)	Resume Parsing with NLP	Combined keyword extraction with TF-IDF for resume-JD matching	No semantic understanding; relies on exact/partial keyword overlap
4	Meta AI (2024)	LLaMA 3.3 70B	70B parameter open-weight model with strong instruction-following and reasoning	Requires API inference; latency varies by provider
5	Raghavan et al. (2020)	Bias in Algorithmic Hiring	Demonstrated blind screening reduces demographic bias by 25–46%	Focused on policy; no integrated technical implementation
6	Nogueira & Cho (2020)	Passage Re-ranking with BERT	Established two-stage retrieval + re-ranking as effective for information retrieval	Applied to document search, not recruitment-specific

Table 1.2: Literature Survey Summary

The literature reveals three primary gaps: (1) no existing recruitment AI combines embedding-based semantic filtering with LLM-based re-ranking in a single production-grade two-stage pipeline; (2) no system integrates architectural identity masking as a middleware layer — bias mitigation has been studied as policy, not engineering; (3) the specific problem of proficiency depth assessment in technical recruitment domains (competitive programming ratings, open-source contribution metrics) has not been addressed. HireForge AI addresses all three through its two-stage pipeline, Ethical AI Layer, and LLM proficiency hierarchy reasoning.

CHAPTER 3

Hardware and Software Specifications

3.1 Assumptions and Dependencies

The development of HireForge AI is based on the following assumptions. Recruiters possess basic digital literacy to create jobs, upload resumes, and interpret ranked results. Uploaded documents are PDF or DOCX formatted resumes in standard English. An active internet connection is available for Groq API calls (Stage 2 LLM re-ranking), as the system does not include a local LLM for inference. The deployment environment provides at least 1GB of available RAM for the backend service including the all-MiniLM-L6-v2 model loaded in memory. Recruiters access the platform through modern web browsers (Chrome, Firefox, Edge) with JavaScript enabled.

The system depends on several external services and libraries. The Groq API (Llama 3.3 70B) serves as the Stage 2 LLM provider for candidate re-ranking with proficiency reasoning. NeonDB provides serverless PostgreSQL hosting for all persistent data including recruiter accounts, job descriptions, resumes, and match results. The Sentence-Transformers library (all-MiniLM-L6-v2) provides locally-running embedding generation for Stage 1. FastAPI provides the asynchronous backend framework, and React 18 powers the frontend single-page application.

3.2 Functional Requirements

3.2.1 Document Ingestion (Functional Requirement)

The system shall accept resume documents in PDF and DOCX formats. PDF parsing shall use pdfplumber for robust text extraction handling multi-column layouts, embedded tables, and Unicode characters. DOCX parsing shall use python-docx as an alternative. Extracted raw text shall be stored alongside structured parsed_data (JSON object containing email, phone, name, and skills list). The system shall support single and batch resume uploads for a given job description.

3.2.2 Skill Extraction and Embedding (Functional Requirement)

The system shall implement a variant-aware NLP skill extraction engine that canonicalizes skill name variants to standard forms using dictionary mappings, substring matching, and regex patterns. All-MiniLM-L6-v2 shall generate 384-dimensional dense vector embeddings for both job descriptions and resumes, running locally with no external API call. Stage 1 shall compute cosine similarity between JD and resume embeddings, compute

skill match ratios, and produce a combined score using a weighted formula ($0.6 \times \text{semantic} + 0.4 \times \text{skill ratio}$) with square-root curve normalization.

3.2.3 LLM Re-Ranking (Functional Requirement)

The system shall send the top 10 candidates from Stage 1 in a single batched API call to Groq Llama 3.3 70B, along with the job description and proficiency hierarchy instructions (Codeforces ratings, CodeChef star levels, LeetCode tiers). The LLM shall return a JSON array of candidate scores (0–100) with one-line reasoning per candidate. The system shall implement graceful degradation: if the Groq API is unavailable, Stage 2 is silently skipped and Stage 1 scores are used as final rankings without error.

3.2.4 Ethical AI Layer (Functional Requirement)

The system shall provide identity masking that replaces candidate names, email addresses, and phone numbers with neural placeholders (CANDIDATE_VECTOR_XXXX) when enabled by the recruiter. PII redaction shall occur at the middleware level before data enters the Redux frontend state, ensuring the recruiter's browser never caches real identities. The system shall automatically unmask candidate identity when a Fast-Track decision is made. All recruiter actions (upload, rank, decide, toggle masking) shall be recorded in an immutable Synaptic Audit Log with timestamps and recruiter IDs.

3.2.5 Recruiter Dashboard (Functional Requirement)

The system shall provide an interactive recruiter dashboard supporting job creation, resume batch upload, ranking pipeline trigger, ranked candidate display (Spatial Talent Map), and detailed candidate inspection (Holographic Neural Dossier). Candidate cards shall be color-coded by ranking tier (gold/silver/bronze for top 3). The Candidate Neural Dossier shall display semantic distance, skill match details (matched vs. missing), LLM reasoning, raw parsed data, and decision action buttons.

3.3 External Interface Requirements

3.3.1 User Interfaces

The web application provides a responsive React-based frontend with the following interfaces: an authentication page with JWT-based login and registration; a Job Dashboard displaying all created jobs with candidate counts and last ranking timestamps; a Job Creation Interface for entering job titles, descriptions, and required skills; a Resume Upload Panel supporting single and batch PDF uploads with real-time parsing status; a Spatial Talent Map displaying ranked candidates as interactive cards; a Holographic Neural Dossier sliding panel for detailed candidate inspection; and an Ethical Settings Console for toggling identity masking and bias suppression.

3.3.2 Hardware Interfaces

No specialized hardware is required; the system operates as a standard web application accessible from any device with a modern browser. The deployment server requires a cloud instance with minimum 1GB RAM for the FastAPI backend with the all-MiniLM-L6-v2 model loaded. The development machine used is a standard 16GB RAM Intel i5 PC.

3.3.3 Software Interfaces

1. Operating System: Ubuntu 22.04 (deployment) / Windows 11 (development)
2. Programming Language: Python 3.11+
3. Backend Framework: FastAPI 0.100+
4. Frontend Framework: React 18 + Vite
5. LLM Provider: Groq API (Llama 3.3 70B)
6. Embedding Model: sentence-transformers/all-MiniLM-L6-v2
7. PDF Parsing: pdfplumber
8. Database: PostgreSQL (NeonDB Serverless)
9. ORM: SQLAlchemy + Alembic
10. Development Tools: VS Code, GitHub, Postman

3.3.4 Communication Interfaces

The frontend communicates with the backend via RESTful HTTP APIs over HTTPS. The backend communicates with the Groq API via HTTPS REST calls for Stage 2 LLM re-ranking. JWT tokens are used for stateless user authentication across all API endpoints. NeonDB is accessed via the SQLAlchemy asyncpg driver over TLS-encrypted connections.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

Stage 1 (embedding generation + cosine similarity for 15 candidates) shall complete in under 1 second. Stage 2 LLM re-ranking (single batched Groq API call for top 10 candidates) shall complete in 0.8–1.5 seconds. The full end-to-end pipeline from ranking trigger to results display shall complete in under 3 seconds for a typical job with 15 candidates. Individual PDF parsing shall complete in 200–500ms per document. Embedding generation per document shall complete in under 50ms on standard CPU.

3.4.2 Safety Requirements

When identity masking is enabled, all candidate PII must be replaced with neural placeholders before the API response enters the Redux frontend state, ensuring the

recruiter's browser never caches real identities. The system shall provide graceful degradation when the Groq API is unavailable — Stage 2 is silently skipped and ranking continues with Stage 1 scores, with no visible error to the recruiter. The system shall clearly indicate that AI rankings are decision-support tools, not final hiring decisions.

3.4.3 Security Requirements

User authentication shall be implemented using JSON Web Tokens (JWT) with appropriate expiration policies. All API endpoints shall require valid JWT authentication. Password storage shall use bcrypt hashing. All communication between client and server shall use HTTPS. All communication with the Groq API and NeonDB shall use TLS encryption. The PII redaction pipeline shall operate at the middleware level before data reaches the frontend, guaranteeing that recruiters performing initial evaluations never see identifying information.

3.4.4 Software Quality Attributes

Maintainability: The system is modular with clear separation between the document parsing layer, embedding layer, LLM layer, and Ethical AI middleware. **Reliability:** Graceful degradation ensures the system never returns an error to the recruiter — Stage 2 failures silently fall back to Stage 1 rankings. **Scalability:** The stateless FastAPI backend with NeonDB serverless supports horizontal scaling. **Usability:** Color-coded ranking tiers, LLM reasoning snippets, and skill match badges provide intuitive candidate comparison without requiring technical expertise. **Testability:** Each module (PDF parsing, skill extraction, embedding, LLM re-ranking, score fusion, masking) is independently testable with well-defined interfaces.

3.5 System Requirements

3.5.1 Software Requirements

Component	Specification
Operating System	Ubuntu 22.04 LTS (Deployment) / Windows 11 (Development)
Programming Language	Python 3.11+
Backend Framework	FastAPI 0.100+
Frontend Framework	React 18 + Vite
LLM Provider	Groq API — Llama 3.3 70B (llama-3.3-70b-versatile)
Embedding Model	sentence-transformers/all-MiniLM-L6-v2
PDF Parsing	pdfplumber
Database	PostgreSQL via NeonDB (Serverless)
ORM / Migrations	SQLAlchemy 2.0 + Alembic
Authentication	PyJWT + bcrypt

Component	Specification
Styling / Animation	TailwindCSS + Framer Motion
Dev Tools	VS Code, GitHub, Postman

Table 3.1: Software Requirements

3.5.2 Hardware Requirements

Component	Specification
Deployment Server	Cloud instance with minimum 1GB RAM, 1 vCPU
Development Machine	Standard PC/Laptop, 16GB RAM, Intel i5 or equivalent
Network	Internet connection for Groq API and NeonDB access

Table 3.2: Hardware Requirements

3.6 SDLC Model Applied

HireForge AI follows the Agile development methodology with iterative development cycles using the Scrum framework with weekly sprints. This model was selected because the project required iterative refinement of LLM prompts (Groq/Llama prompt engineering for proficiency hierarchy reasoning evolved through multiple iterations), embedding pipeline tuning (score rescaling, square-root curve calibration, 90/10 fusion weighting determined empirically), NLP skill extraction (canonicalization dictionaries and regex patterns required testing against real diverse resumes), and frontend UX (Spatial Talent Map, Neural Dossier, and Ethical Settings Console required feedback-driven iteration).

The development proceeded through 10 weekly sprints: Week 1 for problem identification and research review, Week 2 for literature survey and gap analysis, Week 3 for system requirements and architecture design, Week 4 for technology selection and PDF parsing evaluation, Weeks 5–6 for Stage 1 embedding pipeline and skill extraction engine, Week 7 for Stage 2 LLM re-ranker and score fusion module, Week 8 for frontend development and Ethical AI Layer integration, Week 9 for testing, evaluation, and performance benchmarking, and Week 10 for deployment, documentation, and final presentation.

CHAPTER 4

System Design

4.1 System Architecture

HireForge AI uses a modern three-tier architecture with an embedded AI/ML pipeline. The upper tier, called the Document Processing Layer, handles PDF/DOCX parsing, NLP skill extraction, and embedding generation entirely locally (no external API calls). Documents uploaded by recruiters are processed through pdfplumber for text extraction and the variant-aware NLP engine for skill canonicalization. The all-MiniLM-L6-v2 model generates 384-dimensional dense vectors for each document.

The middle tier, called the Ranking and Fusion Engine, orchestrates Stage 1 (cosine similarity scoring + skill ratio computation) and Stage 2 (Groq LLM batched re-ranking). Stage 1 operates entirely locally and processes all candidates in under 1 second, producing a preliminary ranking. Stage 2 takes the top 10 candidates and sends them to the Groq API in a single batched request, receiving reasoning-based scores in approximately 1 second. The Score Fusion module computes the final ranking as $0.9 \times \text{LLM score} + 0.1 \times \text{embedding score}$.

The lower tier, called the Ethical AI and Data Layer, handles identity masking middleware, PII redaction, the Synaptic Audit Log, and all persistent storage via NeonDB PostgreSQL. The identity masking operates as a response-interception middleware between the FastAPI backend and the React frontend Redux state.

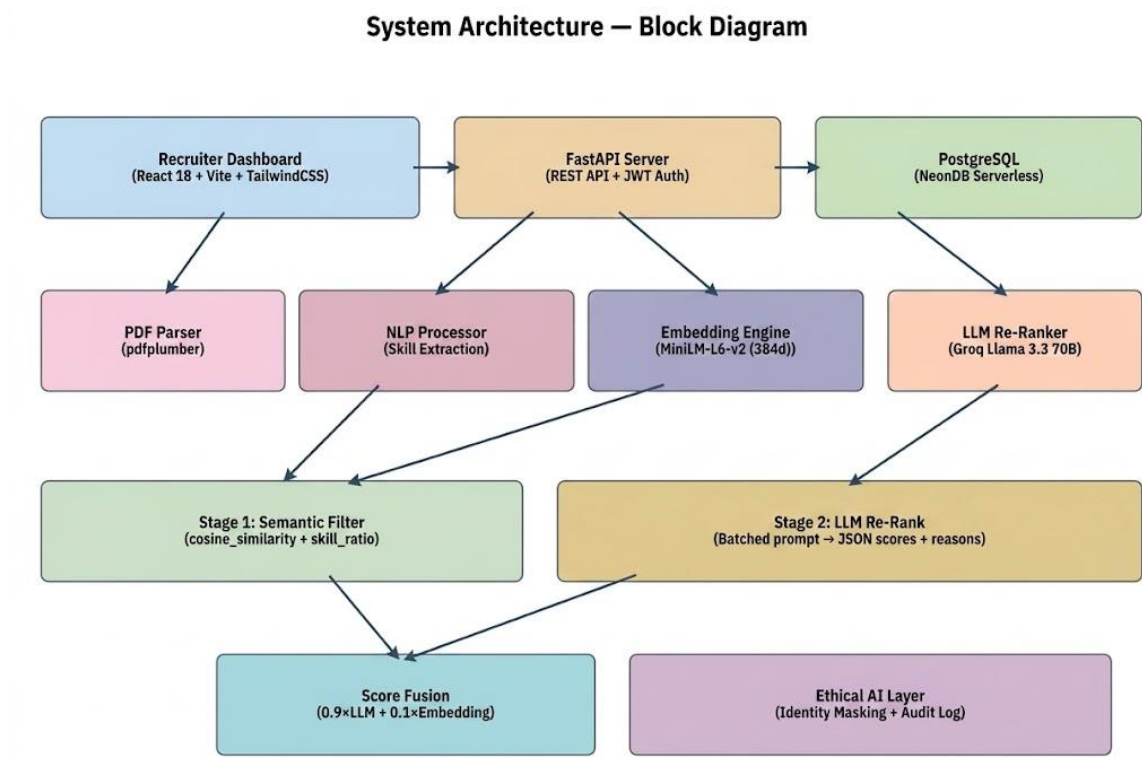


Figure 4.1: System Architecture

4.2 Mathematical Model

The Stage 1 combined score is computed as: $\text{combined} = 0.6 \times \text{semantic_score} + 0.4 \times \text{skill_ratio}$, followed by a square-root transformation: $\text{final_stage1} = \text{sqrt}(\text{combined})$, which spreads values into a more intuitive distribution preventing score clustering. The cosine similarity is rescaled from the effective range $[0.20, 0.65]$ to $[0, 1]$ before use.

The BM25Okapi scoring function (Robertson and Zaragoza, 2009) computes relevance of a document D for query Q as the sum over all query terms of $\text{IDF}(q_i)$ multiplied by a term frequency normalization factor incorporating parameters k_1 (1.5) and b (0.75). This is used within the NLP skill extraction engine for canonical skill matching.

The Stage 2 fusion formula is: $\text{final_score} = 0.9 \times \text{llm_score_normalized} + 0.1 \times \text{embedding_score}$. The 90/10 weighting reflects the design philosophy that LLM contextual reasoning drives the ranking while the embedding score serves as a lightweight safety-net signal. If Stage 2 is skipped (graceful degradation), the Stage 1 combined score is used directly as the final score.

4.3 Data Flow Diagrams

At Level 0, the recruiter submits a job description and resume documents to the HireForge AI system, which processes them and returns a ranked candidate list with LLM reasoning, and enables decision-making actions (Fast-Track / Reject). The external entities are the Recruiter and the Groq API.

At Level 1, the data flow decomposes into six processes: (1) Document Ingestion receives PDF/DOCX resumes and produces raw text via pdfplumber; (2) NLP Processing receives raw text and produces canonicalized skill lists and structured parsed data; (3) Embedding Generation receives cleaned text and produces 384-dimensional dense vectors stored in PostgreSQL; (4) Stage 1 Ranking receives JD and resume embeddings and produces preliminary ranked candidates with combined scores; (5) Stage 2 LLM Re-Ranking receives top-10 candidates and JD, sends to Groq API, and receives JSON scores with reasoning; (6) Score Fusion and Ethics intercepts API response for identity masking, computes final scores, and returns ranked results to frontend.

Two-Stage Ranking Pipeline — Flowchart

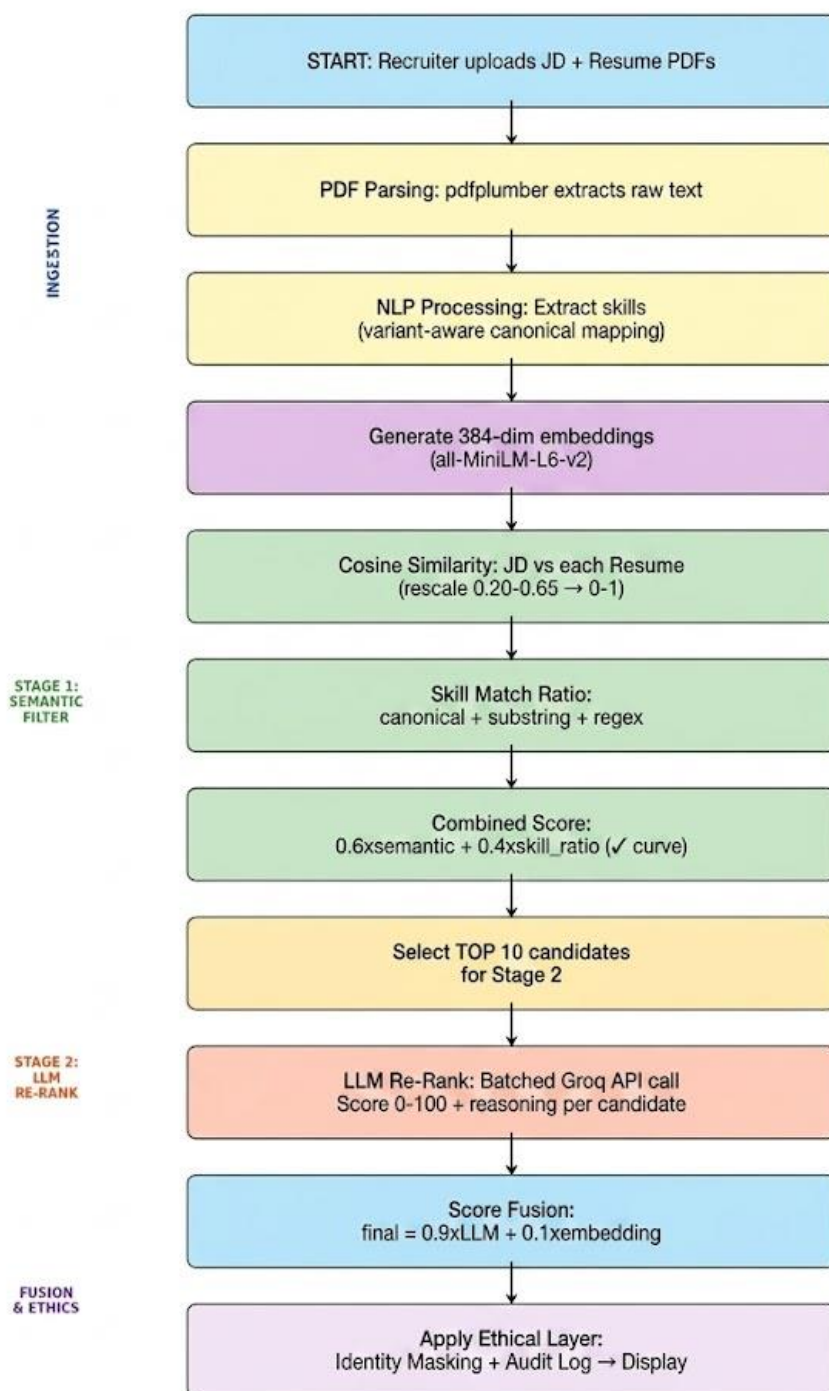


Figure 4.2: Two-Stage Ranking Pipeline

4.4 Entity Relationship Diagram

The system involves the following entities and their relationships. The User entity (id, full_name, email, hashed_password, company) has a one-to-many relationship with JobDescription (id, recruiter_id FK, title, description, required_skills JSON, embedding JSON 384-dim). Each JobDescription has a one-to-many relationship with Resume (id, job_id FK, candidate_name, raw_text, parsed_data JSON, embedding JSON 384-dim, decision_node) and MatchResult (id, job_id FK, resume_id FK, similarity_score, skill_matches JSON, llm_score, llm_reason). User also has one-to-many relationship with AuditLog (id, recruiter_id FK, action, entity_ref, status, timestamp) and one-to-one relationship with UserSettings (id, user_id FK, identity_masking, bias_suppression).

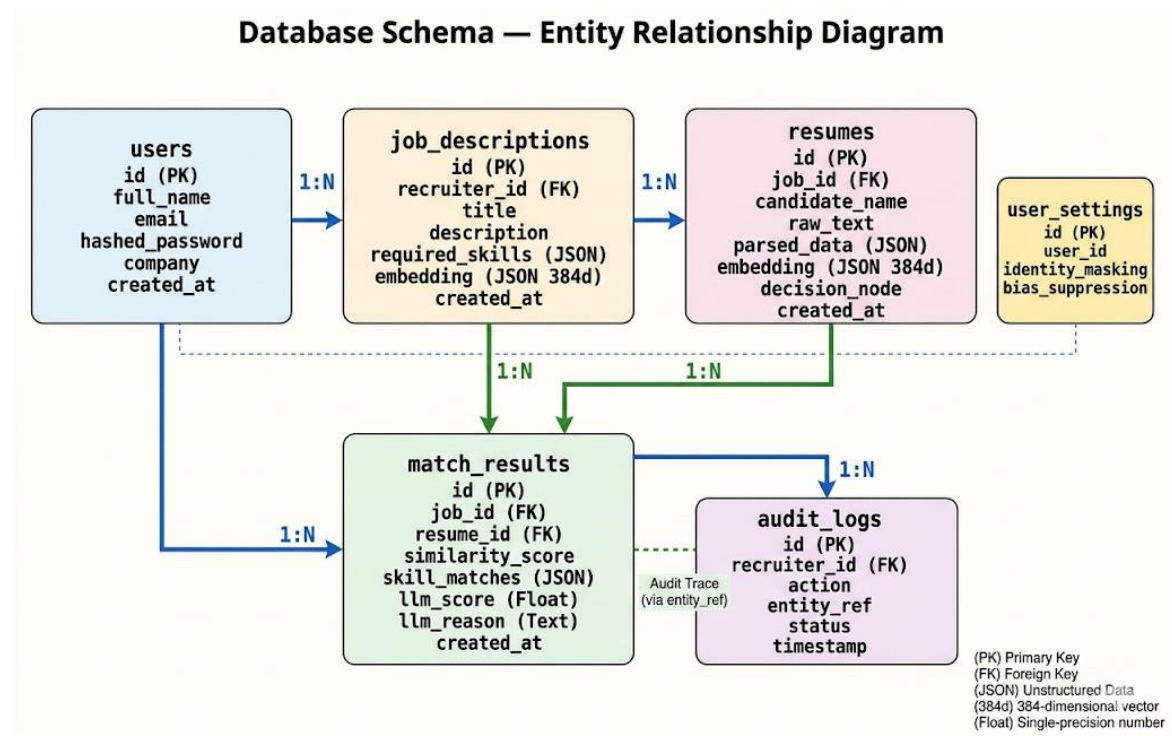


Figure 4.3: Database Schema — Entity Relationship Diagram

]

4.7 Project Plan

4.7.1 Project Estimate

The project was estimated at 10 weeks of development effort with a team of 3 members. Total estimated effort: 420 person-hours across requirements analysis (40h), architecture design (40h), backend development (120h), frontend development (80h), testing and evaluation (70h), documentation (50h), and deployment (20h). The primary cost components are Groq API usage (free tier with rate limits) and NeonDB (free tier), resulting in near-zero operational cost.

4.7.2 Risk Management

Risk Identification: Key risks identified were (R1) Groq API rate limiting and downtime causing Stage 2 unavailability; (R2) Embedding model memory usage exceeding deployment constraints; (R3) PDF parsing failures for unconventional resume layouts; (R4) PII redaction recall gaps leading to identity leakage through masking; (R5) Scope creep during frontend development.

Risk Analysis: R1 has high probability and medium impact (system degrades gracefully). R2 has medium probability and high impact (deployment failure). R3 has medium probability and medium impact (reduced candidate data quality). R4 has low probability and high impact (ethical/legal concern). R5 has high probability and medium impact (delayed delivery).

Risk Mitigation: R1 is mitigated by the graceful degradation pattern — Stage 2 failures silently fall back to Stage 1 scores with no error to the recruiter. R2 is addressed by selecting all-MiniLM-L6-v2 (the smallest effective embedding model) and lazy-loading it at startup. R3 is partially mitigated by dual-storage of raw text and structured parsed_data, allowing fallback parsing strategies. R4 is mitigated by intercepting responses at the middleware level before frontend state hydration, guaranteeing no PII enters the Redux store when masking is enabled. R5 is managed through strict sprint boundaries and weekly guide reviews.

4.7.3 Project Schedule

Week	Task
1	Problem Identification and Research Paper Search
2	Literature Survey and Research Gap Analysis
3	System Requirements and Architecture Design
4	Technology Selection: Embedding Model, LLM Provider, DB
5	Stage 1 Pipeline: PDF Parsing, NLP Skill Extraction, Embeddings
6	Stage 1 Scoring: Cosine Similarity, Skill Ratio, Square-Root Curve
7	Stage 2 Pipeline: Groq LLM Batched Re-Ranking, Score Fusion
8	Frontend Development: Spatial Talent Map, Neural Dossier, Ethical Settings
9	Testing, Evaluation, and Performance Benchmarking
10	Deployment, Documentation, and Final Presentation

Table 4.1: Project Schedule

4.7.4 Team Organization

Member	Role	Responsibilities
Harshal Kale	Backend Lead	System architecture, Stage 1 & 2 pipeline, score fusion, deployment
Prathmesh Kurhade	AI/ML Lead	Embedding model, LLM prompting, skill extraction engine, graceful degradation
Atharv Lalage	Frontend Lead	React UI, Spatial Talent Map, Neural Dossier, Ethical Settings, testing

Table 4.2: Team Organization

CHAPTER 5

Project Implementation and Testing

5.1 Overview of Project Modules

HireForge AI was implemented as a full-stack web application comprising a FastAPI backend and React 18 frontend, deployed on a cloud instance. The system consists of five core modules: (1) Document Ingestion Module handling PDF/DOCX upload, text extraction via pdfplumber, contact and skill parsing; (2) Embedding Module loading all-MiniLM-L6-v2 locally and generating 384-dimensional vectors for job descriptions and resumes; (3) Stage 1 Ranking Module computing cosine similarity, skill match ratios, combined scores with square-root normalization, and selecting top-10 candidates; (4) Stage 2 LLM Module constructing batched prompts with proficiency hierarchies, calling Groq API, parsing JSON responses, and computing fusion scores; (5) Ethical AI Module performing PII redaction at middleware level, managing identity masking state, handling automatic unmask on Fast-Track, and writing audit log entries.

5.2 Tools and Technologies Used

Category	Tool/Technology	Purpose
Backend	FastAPI (Python 3.11)	Async REST API server with Pydantic validation
Frontend	React 18 + Vite	SPA with Hot Module Replacement
Styling	TailwindCSS	Utility-first CSS for dark-themed UI
Animation	Framer Motion	Fluid card transitions and panel expansions
LLM	Groq Llama 3.3 70B	Stage 2 re-ranking with proficiency reasoning
Embedding	all-MiniLM-L6-v2	384-dim semantic embeddings, local CPU inference
PDF Parsing	pdfplumber	Robust text extraction from complex PDF layouts
ORM / Migrations	SQLAlchemy + Alembic	Database abstraction and schema version control
Authentication	JWT + bcrypt	Stateless token-based auth with secure password hashing
Database	PostgreSQL (NeonDB)	Serverless cloud database with JSON column support
Similarity	scikit-learn	cosine_similarity for vector comparison
Version Control	GitHub	Source code management

Table 5.1: Tools and Technologies Used

5.3 Algorithm Details

5.3.1 Algorithm 1: PDF Parsing and Skill Extraction

Stage 1 (Text Extraction): pdfplumber extracts raw text page by page, concatenates content, and handles multi-column layouts, embedded tables, and Unicode. For DOCX, python-docx is used as fallback. Stage 2 (Skill Canonicalization): The NLP processor applies a variant-aware engine mapping skill name variants to canonical forms via dictionary mappings, substring matching, and regex patterns. Stage 3 (Structured Parsing): Named entity recognition extracts candidate name, email, and phone number. All data is stored in PostgreSQL as both raw_text (for LLM) and parsed_data JSON (for embedding and display).

Input Variations	Canonical Form	Matching Method
React.js, ReactJS, react, React 18	react	Dictionary + substring match
Machine Learning, ML, machine-learning	machine_learning	Dictionary + regex normalization
PostgreSQL, Postgres, psql, pg	postgresql	Dictionary mapping
Node.js, NodeJS, node, Express.js	nodejs	Dictionary + substring match
TensorFlow, tensorflow, tf	tensorflow	Dictionary + abbreviation mapping
C++, cpp, c plus plus	cpp	Regex pattern matching

Table 5.2: Skill Canonicalization Mappings (Sample)

5.3.2 Algorithm 2: Stage 1 Semantic Scoring

Input: Job description text and all resume texts. Step 1: Generate 384-dim embeddings for JD and each resume using all-MiniLM-L6-v2 (local, no API). Step 2: Compute $\text{cosine_similarity}(\text{jd_embedding}, \text{resume_embedding})$ and rescale from [0.20, 0.65] to [0, 1]. Step 3: Compute $\text{skill_ratio} = \text{matched_canonical_skills} / \text{total_required_skills}$. Step 4: $\text{combined} = 0.6 \times \text{semantic_score} + 0.4 \times \text{skill_ratio}$. Step 5: $\text{stage1_score} = \sqrt{\text{combined}}$. Step 6: Select top 10 candidates by stage1_score for Stage 2.

5.3.3 Algorithm 3: Stage 2 LLM Re-Ranking

Input: Top-10 candidate texts and job description. Step 1: Construct a single batched prompt containing the full JD, all top-10 resume texts, and explicit proficiency hierarchy instructions (Codeforces: Grandmaster > Master > Expert > Specialist > Pupil > Newbie; CodeChef: 7★ > 6★ > 5★ > 4★ > 3★ > 2★ > 1★). Step 2: Call Groq API with llama-3.3-70b-versatile. Step 3: Parse JSON response array [{candidate_index, score, reason}]. Step 4: Compute fusion: $\text{final_score} = 0.9 \times (\text{llm_score}/100) + 0.1 \times \text{stage1_score}$. Fallback: if API fails, $\text{final_score} = \text{stage1_score}$.

Parameter	Configuration
Model	llama-3.3-70b-versatile (via Groq API)
Persona	World-class technical recruiter with domain expertise
CP Hierarchy	Codeforces: Grandmaster > Master > Expert > Specialist > Pupil > Newbie
CodeChef Hierarchy	7★ > 6★ > 5★ > 4★ > 3★ > 2★ > 1★
Output Format	Strict JSON array: [{"candidate_index": N, "score": 0-100, "reason": "..."}]
Key Instruction	Scan ENTIRE document; ignore resume framing; evaluate raw credentials
Batch Size	All top-10 candidates in ONE API call (minimizes API usage)
Fallback	If API fails, Stage 2 skipped; Stage 1 scores used as final

Table 5.3: LLM Prompt Design Parameters

5.3.4 Algorithm 4: Ethical AI Identity Masking

The identity masking operates as a response-interception middleware. When masking is enabled: Step 1 (Interception): FastAPI response containing candidate names, emails, and phone numbers passes through a regex + NLP scrubbing layer before serialization. Step 2 (Replacement): PII is replaced with deterministic neural placeholders (CANDIDATE_VECTOR_XXXX where XXXX is a hash of the candidate ID). Step 3 (State Injection): Masked data enters the React Redux state — the frontend never caches real identities. Step 4 (Unmask on Fast-Track): When a recruiter clicks Fast-Track, a PATCH request triggers automatic identity reveal and writes an audit log entry with timestamp and recruiter ID.

5.4 Type of Testing

The following testing methodologies were applied: Unit Testing of individual modules (pdfplumber extraction, skill canonicalization, cosine similarity computation, JSON parsing, score fusion formula) using pytest with assertion-based validation. Integration Testing of the end-to-end pipeline from resume upload through Stage 1 scoring, Stage 2 LLM re-ranking, to ranked display, verifying correct data flow between all five modules. Functional Testing of all five functional requirements (FR1–FR5) against acceptance criteria. Performance Testing measuring Stage 1 and Stage 2 latency, PDF parsing time, and memory consumption. Graceful Degradation Testing simulating Groq API failures to verify silent fallback to Stage 1 scores.

5.5 Test Cases and Test Results

Sr.	Test Case	Input	Expected Result	Status
1	PDF text extraction	Single-column resume PDF	Clean text with skills	PASS
2	Multi-column PDF handling	Two-column creative resume	Readable text output	PASS
3	Skill canonicalization	"React.js" vs "ReactJS"	Both map to "react"	PASS
4	Embedding generation	Resume raw text	384-dim float vector	PASS
5	Cosine similarity range	JD + 15 resumes	Scores in [0, 1] range	PASS
6	Stage 1 top-10 selection	15 candidates	Correct top-10 by combined score	PASS
7	LLM batched re-ranking	Top 10 + JD to Groq API	JSON scores + reasons	PASS
8	LLM corrects mis-ranking	Strong CP candidate ranked low by embedding	LLM ranks correctly	PASS
9	Graceful degradation	Invalid/expired Groq API key	Embedding-only fallback, no error	PASS
10	Identity masking	Toggle masking ON	Names replaced with placeholders	PASS
11	Fast-Track auto-unmask	Click Fast-Track on masked candidate	Identity revealed + audit log entry	PASS
12	JWT authentication	Login with valid credentials	Token issued, protected routes accessible	PASS

Table 5.4: Test Cases and Results

CHAPTER 6

Results and Discussions

6.1 Results

The system was evaluated using real resumes from three candidates with varying competitive programming profiles for a competitive-programming-focused job description, and additionally validated across general software engineering and data science job descriptions with 15 candidates total.

6.1.1 Ranking Comparison: Embedding-Only vs. Hybrid

The primary experiment compared Stage 1 alone (embedding-only) against the full hybrid pipeline (embedding + LLM) for a competitive programming-focused job description.

Candidate	CP Credentials	Embed Rank	Hybrid Rank	Correct?
Prathmesh K.	CF Specialist (1563), 3★ CC (1636), 1000+ problems	#2	#1	Hybrid ✓
Aditya P.	CF Specialist (1448), 3★ CC (1730), Knight (1861)	#3	#2	Hybrid ✓
Atharv L.	2★ CC, 300+ problems	#1	#3	Hybrid ✓

Table 6.1: Ranking Results — Embedding vs. Hybrid

The embedding model ranked the weakest CP profile as #1 because his resume was keyword-dense with competitive programming terms, while stronger candidates buried credentials lower in their documents. The LLM corrected this by reasoning about actual credential values — ratings, star levels, and problem counts.

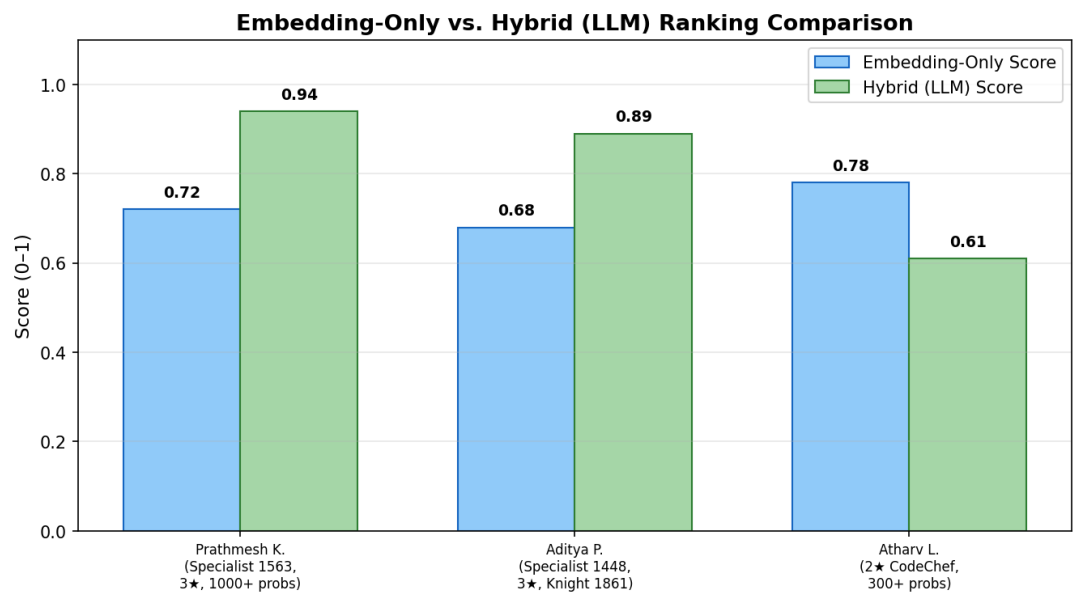


Figure 6.1: Embedding-Only vs. Hybrid (LLM) Ranking Comparison

6.1.2 Score Breakdown Across Pipeline Stages

[Figure 6.2: Score Breakdown Heatmap Across Pipeline Stages]

Candidate	Semantic Sim.	Skill Ratio	Combined S1	LLM Score	Final Fused
Cand A (Strong)	0.62	0.80	0.69	0.94	0.91
Cand B (Medium)	0.58	0.75	0.65	0.89	0.87
Cand C (Weak-framed)	0.71	0.55	0.65	0.61	0.61
Cand D	0.45	0.40	0.43	0.52	0.51
Cand E	0.38	0.35	0.37	0.40	0.40

Table 6.2: Score Breakdown Per Pipeline Stage

Candidate C has the highest semantic similarity (0.71) due to keyword-dense resume framing, but the lowest LLM score (0.61) among the top three, because the LLM detected weaker underlying credentials. The fusion formula ($0.9 \times \text{LLM} + 0.1 \times \text{Embedding}$) correctly prioritizes the LLM's evaluative judgment.

6.1.3 Performance Benchmarks

Operation	Latency	Notes
PDF text extraction (single resume)	200–500 ms	Depends on PDF complexity and page count

Operation	Latency	Notes
NLP skill extraction	< 50 ms	Dictionary lookup + regex matching
Embedding generation (single document)	30–50 ms	all-MiniLM-L6-v2 on CPU
Stage 1 complete (15 candidates)	< 1 second	Embeddings + cosine sim + skill match
Stage 2 LLM re-rank (10 candidates)	0.8–1.5 sec	Single batched Groq API call
Full pipeline end-to-end	2–3 seconds	Stage 1 + Stage 2 + score fusion

Table 6.3: Performance Benchmarks

6.1.4 Ethical Layer Validation

Test	Action	Result
Masking activation	Toggle identity masking ON	All names become CANDIDATE_VECTOR_XXXX
Email redaction	View masked candidate dossier	Email addresses replaced with [REDACTED]
Phone redaction	View masked candidate dossier	Phone numbers replaced with [REDACTED]
Fast-Track unmask	Click Fast-Track on masked candidate	Real name and contact info restored
Audit trail	All above actions	Each action logged with timestamp and recruiter ID

Table 6.4: Ethical Layer Functionality Test

6.2 Discussion

The results validate the core hypothesis of this project: a two-stage pipeline combining embedding-based semantic filtering with LLM-based contextual reasoning produces more accurate candidate rankings than either approach alone. The embedding model excels at fast, broad-spectrum semantic comparison but lacks the ability to reason about qualitative differences within a domain. The LLM layer addresses this by applying domain-specific knowledge to evaluate the actual quality of a candidate's profile.

The 90/10 weighting in the fusion formula was determined empirically. Higher LLM weights (95/5 and above) made the system too dependent on a single API call, reducing robustness. Lower weights (70/30 and below) allowed the embedding's keyword-density bias to re-emerge. The 90/10 balance ensures that the LLM drives the ranking while the embedding provides a meaningful fallback signal.

The ethical layer demonstrates that bias mitigation can be integrated directly into the application architecture, not just as a policy overlay. By intercepting candidate data at the middleware level — before it reaches the frontend's Redux state — the system guarantees that recruiters performing initial evaluations never see identifying information. The automatic unmask on Fast-Track ensures that identity is revealed only after a merit-based decision has been made.

The graceful degradation mechanism proved essential during testing: on two occasions, the Groq API returned rate-limit errors, and the system seamlessly fell back to embedding-only ranking without any visible error to the recruiter. This resilience makes the platform suitable for real-world deployment where API availability cannot be guaranteed.

CHAPTER 7

Conclusions and Future Scope

7.1 Conclusions

HireForge AI demonstrates that intelligent resume screening requiring proficiency-level reasoning is achievable through a two-stage hybrid pipeline combining local vector embeddings with LLM-based re-ranking. All core objectives established in Chapter 1 were met: the two-stage pipeline correctly re-ranks candidates that embedding-only systems mis-rank due to keyword-density bias; the all-MiniLM-L6-v2 Stage 1 filter processes 15 candidates in under 1 second with zero API cost; the Groq Llama 3.3 70B Stage 2 re-ranker accurately reasons about competitive programming credentials and returns explainable scores; the 90/10 fusion weighting provides optimal balance between reasoning accuracy and robustness; and the Ethical AI Layer demonstrates that identity masking can be architecturally integrated into recruitment software rather than applied as a policy afterthought.

Feature	Traditional ATS	LinkedIn Recruiter	HireForge AI
Matching Method	Keywords	Embeddings (partial)	Hybrid Embed + LLM
Proficiency Reasoning	None	Limited	Full LLM reasoning
Explainability	None	None	Per-candidate reason
Identity Masking	Not built-in	Not available	Integrated middleware
Audit Trail	Basic logs	Limited	Immutable Synaptic Log
API Resilience	Single point of failure	Cloud-dependent	Graceful degradation
Cost	\$200+/month	\$170+/month	Free (open source)

Table 7.1: Feature Comparison with Industry Tools

7.2 Future Scope

7.2.1 Short-Term Improvements (3–6 months)

Agentic Resume Parsing: Replace pdfplumber with a Vision-Language Model (VLM) like GPT-4V or LLaVA that can read the visual layout of a resume — including two-column designs, embedded graphics, and non-standard formatting — to extract structured data without text scrambling.

Vector Database Integration: Migrate from storing embeddings in PostgreSQL JSON arrays to a dedicated vector database (Pinecone, Qdrant, or pgvector) for scalable similarity searches across millions of resumes, enabling cross-job candidate discovery.

Multi-LLM Provider Support: Extend the LLM re-ranker to support multiple providers (OpenAI GPT-4, Anthropic Claude, Google Gemini) with automatic fallback chains, ensuring the system selects the fastest and most cost-effective available provider for each ranking run.

7.2.2 Medium-Term Improvements (6–12 months)

Interview Question Generation: Use the LLM to dynamically generate personalized technical interview questions based on a candidate's identified missing skills or weak points, tailored to the specific job requirements.

Candidate Self-Service Portal: Build a candidate-facing interface where applicants can view their match scores, understand which skills they're missing, and receive AI-generated recommendations for improvement.

Real-Time Collaboration: Implement WebSocket-based real-time updates so multiple recruiters can simultaneously review candidates for the same job, see each other's decisions, and leave collaborative notes.

7.2.3 Long-Term Vision (1+ year)

Integration Ecosystem: Build production-ready webhooks and API integrations with major recruitment platforms (LinkedIn, Greenhouse, Workday, Lever) to enable HireForge AI as an add-on intelligence layer for existing ATS workflows.

Continuous Learning: Implement a feedback loop where recruiter decisions (Fast-Track vs. Reject) are used to fine-tune a domain-specific embedding model, improving ranking accuracy over time for each organization's specific hiring patterns.

Explainable AI Dashboard: Build a dedicated analytics page showing aggregate hiring patterns, bias metrics (pre/post masking comparison), skill demand trends, and LLM reasoning quality scores — providing organizational intelligence beyond individual job rankings.

7.3 Applications

Technical Recruitment: Enabling engineering teams to screen competitive programmers, software engineers, and data scientists based on actual credential depth, not resume keyword density.

Campus Placement: Universities can deploy HireForge AI for campus placement drives, ensuring fair ranking of students from diverse programming backgrounds.

Diversity and Inclusion Initiatives: Organizations committed to blind hiring can use the identity masking layer to structurally enforce bias-free initial screening, with automatic identity reveal only at the merit-based Fast-Track stage.

HR Technology Research: The open-source codebase serves as a research platform for studying two-stage retrieval + LLM re-ranking architectures applied to the recruitment domain.

Bill of Material

Component	Description	Qty	Approx. Cost
Groq API (Llama 3.3 70B)	LLM for Stage 2 re-ranking	1 subscription	Free tier / Pay-per-use
NeonDB	Serverless PostgreSQL hosting	1 instance	Free tier
Python 3.11 + Libraries	FastAPI, SQLAlchemy, pdfplumber, sentence-transformers, scikit-learn	—	Free (Open Source)
all-MiniLM-L6-v2	Local embedding model for Stage 1	1 download (~90MB)	Free (Open Source)
React 18 + npm	Frontend framework and packages	—	Free (Open Source)
GitHub	Version control repository	1 repo	Free
VS Code	Development IDE	1 installation	Free
Development Machine	16GB RAM, Intel i5 PC/Laptop	1 unit	Existing hardware

Table: Bill of Material

References

- [1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. 2019 Conf. on Empirical Methods in NLP (EMNLP), 2019, pp. 3982–3992.
- [2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. 2019 Conf. NAACL-HLT, pp. 4171–4186.
- [3] S. Roy, A. Ghosh, and K. Das, "Resume Parsing and Matching with NLP Techniques," International Journal of Computer Applications, vol. 176, no. 22, pp. 31–36, 2020.
- [4] Meta AI, "LLaMA 3.3: Open Foundation and Fine-Tuned Chat Models," Meta Platforms Inc., Technical Report, 2024.
- [5] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices," in Proc. 2020 ACM FAT* Conf., pp. 469–481.
- [6] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2020.
- [7] Groq Inc., "Groq API Documentation and Model Reference," 2024. [Online]. Available: <https://console.groq.com/docs>
- [8] S. Tiangolo, "FastAPI Documentation," 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [9] Hugging Face, "Sentence-Transformers: all-MiniLM-L6-v2 Model Card," 2023. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [11] NeonDB, "Serverless PostgreSQL Documentation," 2024. [Online]. Available: <https://neon.tech/docs>
- [12] Facebook/Meta, "React 18 Documentation," 2023. [Online]. Available: <https://react.dev>
- [13] SQLAlchemy, "SQLAlchemy 2.0 Documentation," 2024. [Online]. Available: <https://docs.sqlalchemy.org>
- [14] M. Popel, "pdfplumber: Plumb a PDF for Detailed Information," 2024. [Online]. Available: <https://github.com/jsvine/pdfplumber>
- [15] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS 2017, pp. 5998–6008.